

Stringhe in Python

Stringhe in Python

Come lavorare con il testo in Python.

Cosa sono le stringhe?

Una **stringa** è semplicemente del testo: un nome, un messaggio, una frase. In Python le stringhe si scrivono tra virgolette singole `'...'` o doppie `"..."` :

```
nome = "Alice"  
saluto = 'Ciao, mondo!'
```

Usa virgolette doppie se dentro il testo c'è un apostrofo, e singole se c'è già una virgoletta doppia:

```
con_apice = "It's a great day"      # apostrofo dentro → virgolette doppie  
fuori  
con_virgolette = 'Disse "ciao!"'    # virgolette doppie dentro → singole  
fuori
```

Per testo su più righe, usa le virgolette triple:

```
poesia = """Questa stringa  
si estende su  
più righe."""
```

Accedere ai singoli caratteri

Come nelle liste, ogni carattere ha una posizione (indice) che parte da `0` :

```
testo = "Python"
print(testo[0])    # P  - primo carattere
print(testo[1])    # y
print(testo[-1])   # n  - ultimo carattere
print(testo[-2])   # o  - penultimo
```

Estrarre una parte di stringa (slicing)

Puoi estrarre una sottostringa indicando da dove a dove:

```
testo = "Python"
print(testo[0:3])   # Pyt  - dalla posizione 0 alla 2 (3 esclusa)
print(testo[2:])    # thon - dalla posizione 2 alla fine
print(testo[:4])    # Pyth - dall'inizio alla posizione 3
print(testo[::-1])  # nohtyP - la stringa al contrario
```

Lunghezza di una stringa

```
testo = "Python"
print(len(testo))  # 6
```

Mettere stringhe insieme

Puoi unire due stringhe con `+` e ripetere una stringa con `*` :

```
a = "Ciao"
b = " mondo"
print(a + b)    # Ciao mondo
print(a * 3)    # CiaoCiaoCiao
```

Le f-string: il modo migliore per inserire variabili

Invece di unire le stringhe con `+` (che è scomodo), usa le **f-string**: scrivi una `f` prima delle virgolette e metti le variabili tra parentesi graffe `{}` :

```
nome = "Alice"
eta = 16
print(f"Mi chiamo {nome} e ho {eta} anni.")
# Mi chiamo Alice e ho 16 anni.

a = 5
b = 3
print(f"La somma di {a} e {b} è {a + b}.")
# La somma di 5 e 3 è 8.
```

Le f-string sono la forma moderna e più leggibile. Usale sempre quando devi mettere variabili nel testo.

Metodi utili delle stringhe

Le stringhe hanno molti strumenti integrati. Ecco i più usati, divisi per categoria.

Cambiare maiuscole e rimuovere spazi

```
testo = "  Ciao, Mondo!  "

print(testo.upper()) # "  CIAO, MONDO!  " - tutto maiuscolo
print(testo.lower()) # "  ciao, mondo!  " - tutto minuscolo
print(testo.strip()) # "Ciao, Mondo!" - rimuove spazi iniziali e finali
```

Cercare, sostituire e dividere

```
testo = "Ciao, Mondo!"

print(testo.replace("Mondo", "Python")) # "Ciao, Python!"

frutta = "mela,banana,ciliegia"
print(frutta.split(",")) # ['mela', 'banana', 'ciliegia'] - divide in lista
print(",".join(["mela", "banana", "ciliegia"])) # "mela,banana,ciliegia"
- unisce
```

Controllare il contenuto

```
testo = "ciao"

print(testo.capitalize()) # "Ciao" - prima lettera maiuscola
print(testo.startswith("ci")) # True - inizia con "ci"?
print(testo.endswith("ao")) # True - finisce con "ao"?
print(testo.find("ia")) # 1 - dove inizia "ia"? (-1 se non trovato)
print(testo.count("a")) # 1 - quante volte compare "a"?
```

Caratteri speciali

Alcune combinazioni di caratteri con `\` hanno un significato speciale:

Sequenza	Significato	Esempio
<code>\n</code>	Nuova riga	<code>"Prima riga\nSeconda riga"</code>
<code>\t</code>	Tabulazione (spazi)	<code>"Nome:\tAlice"</code>
<code>\\</code>	Un singolo backslash	<code>"C:\\Users\\Alice"</code>
<code>\"</code>	Virgolette doppie dentro la stringa	<code>"Disse \"ciao!\""</code>

```
print("Prima riga\nSeconda riga")
# Prima riga
# Seconda riga

print("Nome:\tAlice")
# Nome: Alice
```

Le stringhe non si modificano

Attenzione: le stringhe sono **immutabili**. Non puoi cambiare un singolo carattere. Se hai bisogno di una stringa modificata, devi crearne una nuova:

```
testo = "ciao"
# testo[0] = "C" - ERRORE! Non si può modificare

# Crea invece una nuova stringa
testo = "C" + testo[1:]
print(testo) # Ciao
```